Reporte Técnico: Generación y Carga Masiva de Datos

Objetivo del Documento

El presente informe detalla la ejecución, auditoría y resultados del proceso automatizado de generación y carga masiva de datos transaccionales para el dominio "Bodegas del Atuel". Para garantizar la trazabilidad, las pruebas y la reproducibilidad exacta del entorno, el algoritmo de generación (basado en la librería Faker) ha sido inicializado con una semilla (seed) estática.

Fase 1: Despliegue del Esquema y Carga Masiva

1.1 Inicialización de la Estructura (DDL)

Como paso preliminar, se validó la ejecución íntegra del esquema físico. Tras inicializar el contenedor de la base de datos mediante el orquestador Docker Compose, se inyectó el script principal de migraciones utilizando el siguiente comando:

```
sudo docker exec -i bodegas_db psql -U postgres -d bodegas_atuel < db/migrations/001_esquema.sql
```

```
DROP SCHEMA
CREATE SCHEMA
GRANT
GRANT
CREATE TYPE
CREATE TYPE
CREATE TYPE
CREATE TYPE
CREATE DOMAIN
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
```

Resultado: La ejecución finalizó exitosamente respetando el orden jerárquico de creación. No se registraron errores de sintaxis ni violaciones de dependencias, confirmando que la estructura física está correctamente normalizada.

1.2 Ejecución del Script de Carga Masiva

Con la base de datos limpia y estructurada, se procedió a correr el script orquestador de inserciones. Se midió el rendimiento del proceso utilizando el comando `time`:

```
time uv run db/seed/script_carga.py
```

```
Iniciando carga de datos...
Catálogo cepa cargado correctamente.
Catálogo maridaje cargado correctamente.
Catálogo provincia cargado correctamente.
Catálogo departamento cargado correctamente.
Catálogo localidad cargado correctamente.
Generando datos masivos con Faker (Esto tomará unos segundos)...
Datos transaccionales generados con éxito.
Proceso finalizado. Base de datos poblada exitosamente.

real    1m22.555s
user    0m13.840s
sys     0m8.799s
```

Resultado: El script pobló exitosamente los catálogos estáticos mediante comandos COPY y generó la volumetría transaccional requerida procesando inserciones masivas optimizadas por bloques.

Fase 2: Auditoría y Consultas de Control

Para certificar la consistencia lógica de los datos generados y el estricto cumplimiento de las reglas de negocio, se ejecutaron las siguientes consultas de control a través de la consola interactiva psql.

2.1 Control de Volumetría General

Se verifica que el volumen de los datos generados cumpla con los umbrales mínimos establecidos para el entorno de pruebas.

```
SELECT
    (SELECT COUNT(*) FROM cliente) AS clientes_totales,
    (SELECT COUNT(*) FROM comprobante) AS comprobantes_totales,
    (SELECT COUNT(*) FROM vino_comprobante) AS renglones_totales,
    (SELECT COUNT(*) FROM vino) AS vinos_totales;
```

```
bodegas_atuel=# SELECT
bodegas_atuel-#     (SELECT COUNT(*) FROM cliente) AS clientes_totales,
bodegas_atuel-#     (SELECT COUNT(*) FROM comprobante) AS comprobantes_totales,
bodegas_atuel-#     (SELECT COUNT(*) FROM vino_comprobante) AS renglones_totales,
bodegas_atuel-#     (SELECT COUNT(*) FROM vino) AS vinos_totales;
 clientes_totales | comprobantes_totales | renglones_totales | vinos_totales
------------------+----------------------+-------------------+---------------
            20000 |               150000 |            449515 |           400
(1 row)
```

2.2 Distribución de la Jerarquía de Clientes

Se audita la correcta clasificación de los 20.000 clientes generados a través de la red de subtipos lógicos.

```
SELECT tipo_cliente, COUNT(*) as cantidad
FROM cliente
GROUP BY tipo_cliente;
```

```
bodegas_atuel=# SELECT tipo_cliente, COUNT(*) as cantidad
bodegas_atuel-# FROM cliente
bodegas_atuel-# GROUP BY tipo_cliente;
 tipo_cliente | cantidad
--------------+----------
 socio        |    16059
 distribuidor |     2983
 exportador   |      958
(3 rows)
```

2.3 Validación de Composición de Vinos (Regla de Negocio)

Se comprueba la integridad referencial de los cortes de vino. La suma de los porcentajes de las cepas que componen cada vino debe ser exactamente el 100%. Un retorno de cero (0) filas garantiza la ausencia de anomalías.

```
SELECT codigo_vino, SUM(porcentaje) as total_porcentaje
FROM vino_cepa
GROUP BY codigo_vino
HAVING SUM(porcentaje) != 100;
```

```
bodegas_atuel=# SELECT codigo_vino, SUM(porcentaje) as total_porcentaje
bodegas_atuel-# FROM vino_cepa
bodegas_atuel-# GROUP BY codigo_vino
bodegas_atuel-# HAVING SUM(porcentaje) != 100;
 codigo_vino | total_porcentaje
-------------+------------------
(0 rows)
```

2.4 Validación de Límites Legales de Graduación Alcohólica

Se certifica que ningún vino generado posea una graduación alcohólica fuera de los límites permitidos (5% a 20%).

```
SELECT COUNT(*) as vinos_ilegales
FROM vino
WHERE graduacion < 5 OR graduacion > 20;
```

```
bodegas_atuel=# SELECT COUNT(*) as vinos_ilegales
bodegas_atuel-# FROM vino
bodegas_atuel-# WHERE graduacion < 5 OR graduacion > 20;
 vinos_ilegales
----------------
              0
(1 row)
```

2.5 Auditoría de Consistencia Financiera

Se cruzan los datos de cabecera contra el detalle transaccional. El subtotal registrado en cada comprobante debe coincidir de forma matemáticamente exacta con la sumatoria de sus renglones correspondientes. Un resultado vacío evidencia un estado contable impecable.

```
SELECT c.numero, c.subtotal, SUM(vc.subtotal) as suma_renglones
FROM comprobante c
JOIN vino_comprobante vc ON c.numero = vc.nro_comprobante
GROUP BY c.numero, c.subtotal
HAVING c.subtotal != SUM(vc.subtotal);
```

```
bodegas_atuel=# SELECT c.numero, c.subtotal, SUM(vc.subtotal) as suma_renglones
bodegas_atuel-# FROM comprobante c
IN vino_compbodegas_atuel-# JOIN vino_comprobante vc ON c.numero = vc.nro_comprobante
bodegas_atuel-# GROUP BY c.numero, c.subtotal
bodegas_atuel-# HAVING c.subtotal != SUM(vc.subtotal);
 numero | subtotal | suma_renglones
--------+----------+----------------
(0 rows)
```